

Claude Code 101

Claude code is an agent orchestration that happens to be excellent at coding. Here's how the orchestration works :

1. [Claude.md](#) → Persistent memory and rules
2. Skills → On demand knowledge (Auto or manual call)
3. Slash Commands → Reusable workflow (manual call)
4. Hooks → Deterministic automation
5. Subagents → Parallel specialist
6. MCP Servers → External tools

Why claude Code?

There are many coding tools, but most of them are tools to use, not a system that can be configured like claude code. Claude code operates at project level, Read full codebase, reads the error, fixes them in a loop, it has memory across sessions, automation via hooks, parallelism via subagents, and external integration via MCP.

Let's dig into the architecture.

CLAUDE.md

Claude has a permanent memory about the project. It persists in the memory across sessions. It has a budget of roughly 150-200 instructions while the internal system prompt has about 50 instructions.

How to structure it?

1. WHAT (Codebase Map)
 - a. Tech Stack & Versions
 - b. Directory Structure
 - c. Key Modules & Their Roles
2. WHY (Project Purpose)
 - a. Business and User Context
 - b. Gives Macro context (Claude gives better micro decisions)

3. HOW (Working Instructions)
 - a. Build & Test Instructions
 - b. Code style specifics
 - c. What Claude gets wrong (optional) - if Claude does something wrong.

File Location Hierarchy

Use the hierarchy to optimize.

~/.claude/CLAUDE.md → Applies to all sessions.

./CLAUDE.md → Project root (Commit to git)

./CLAUDE.local.md → Personal override (gitignore)

./src/api/CLAUDE.md → loaded on-demand when working in the dir/

What should I not do?

1. Stuff everything into one file
2. Documenting what Claude already does right
3. Vague instructions
4. [CLAUDE.md](#) to enforced behaviours (it is only an advisory, not deterministic)
5. Make sure file is under 200 lines

HOOKS

A shell command that claude code runs automatically at a specific points in its lifecycle. Claude doesn't decide, it is deterministic.

How do they work?

Define them in .claude/settings.json

ou don't have to watch the terminal.

```
json
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
```

```

    "hooks": [
      { "type": "command", "command": "jq -r '.tool_input.file_path' | xargs npx
prettier --write" }
    ]
  },
  "PreToolUse": [
    {
      "matcher": "Bash",
      "hooks": [
        { "type": "command", "command": "jq -r '.tool_input.command' | grep -qE
'rm -rf|git push.*--force|DROP TABLE' && { echo 'Blocked: dangerous command'
>&2; exit 2; } || exit 0" }
      ]
    },
    {
      "matcher": "Edit|Write",
      "hooks": [
        { "type": "command", "command": "jq -r '.tool_input.file_path' | grep -qE
'\\.env|package-lock\\.json' && { echo 'Blocked: protected file' >&2; exit 2; } || exit
0" }
      ]
    }
  ],
  "Notification": [
    {
      "hooks": [
        { "type": "command", "command": "osascript -e 'display notification
\\\"Claude needs you\\\" with title \\\"Claude Code\\\"'" }
      ]
    }
  ]
}

```

Four most generic and important :

1. Auto-format after edits (**PostToolUse**): the one from earlier.

2. Block dangerous commands (**PreToolUse** on **Bash**): exit code 2 blocks the command and tells Claude why.
3. Protect sensitive files (**PreToolUse** on **Edit|Write**): stops edits to secrets and lockfiles.
4. Desktop notification (**Notification**): alerts you when Claude needs permission or is idle, so you don't have to watch the terminal.

SubAgent

A subagent is a separate Claude instance that the main session gives to. They have their own context protection, new perspective, or maybe you can use a cheaper model.

How to build?

1. Create `.claude/agents/<name>.md` (project).
2. Fill in the frontmatter: name, description, tools, and optionally model. [Likely]
3. Write the system prompt in the body, with its job, a checklist, and the output format. [Certain]
4. Test it by invoking it explicitly: "use the X subagent on Y." [Likely]

Ex:

markdown

name: code-reviewer

description: Reviews code changes for bugs, security, and edge cases. Use after any implementation is complete.

tools: Read, Grep, Glob, Bash

model: sonnet

You are a staff engineer reviewing a diff. Challenge shortcuts.

Check correctness, edge cases, security, and readability.

Output: MUST FIX, SHOULD FIX, CONSIDER.

1. **code-reviewer:** reads a diff in a fresh context and reports bugs, security issues, and edge cases without changing any files.
2. **explorer:** answers "how does X work in this codebase?" using read-only tools, so your main context stays clean.
3. **test-writer:** writes and runs tests for a given module, matching the project's existing test style.
4. **debugger:** reproduces a failure, tests hypotheses, and reports the root cause with evidence instead of applying a fix.
5. **planner:** reads the relevant code and returns a step-by-step implementation plan, without writing any code.

Two-Claude Review Pattern

Session A implement

Session B will give the most honest feedback. Since they dont have any shortcuts on the project or context.

MCP Servers

Claude codes connections to realworld. Can be database, github, internal API, etc.

Adding MCP servers to settings.json,

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "ghp_your_token_here"
      }
    },
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres"],
      "env": {
        "POSTGRES_CONNECTION_STRING": "postgresql://user:pass@localhost/mydb"
      }
    }
  }
}
```

Setup two MCP integration : One is read-only, the other read and write.

For example, you could give your code-reviewer subagent read-only DB access, while implementer subagent gets write access

Skills

/recap → Update all md file (such as readme.md, [claude.md](#), or [design.md](#)) when called after a feature is done or anytime wants to commit (this auto add and commit)

/cleanup → removes deadcode, unused import, debug statements from the current changes

/cleanup_all → same as cleanup, but to check the whole repo

/interview → it ask to use the planner subagent to create PLAN_TASK_NAME.md and explain what it going to do after.

/explain → it ask to use the explore subagent to explain with specific template to me (make it easier for me to understand)

Prompt

Interview me about (always ask claude to keep asking)